Product Requirements Document (PRD)

Fast Logistics — Alpha Release (V4 Integrated)

Contractor / Developer: Elastro Cloud

Target Market: Port Harcourt, Rivers State

Target Date: Q3/Q4 2026

Status: FINAL SPECIFICATION FOR ARCHITECTURE

1. Executive Summary

This document outlines the comprehensive product scope, technical architecture, operational guardrails, and execution timeline for Fast Logistics, a hyper-local delivery and tracking platform engineered by Elastro Cloud. Operating within Port Harcourt for its alpha phase, the ecosystem seamlessly bridges consumers, localized retail vendors, and independent dispatch riders. A core technological differentiator is its low-latency real-time tracking combined with an asynchronous peer-to-peer negotiation pricing framework.

2. Problem Statement & Objectives

Fixed, algorithmic travel pricing paradigms routinely break down under local conditions, failing to reflect micro-realities such as severe traffic spikes, ongoing infrastructure repairs, and varying road surfaces. The objective of Fast Logistics is to establish an ultra-lean, performant codebase that pairs robust geolocation radar arrays with an dynamic bidding engine. By giving both consumers and dispatch operators equal input into fare pricing, the architecture targets higher trip fulfillment metrics and greater market stability.

3. Product Scope & Features

3.1 Customer Application (Flutter)

- **Frictionless Authentication:** Supports a localized guest mode experience via cached local device unique identifiers. Phone numbers and OTP authentication are deferred until final route checkout.
- **Real-Time Bidding Interface:** Empowers users to propose initial delivery target rates, evaluate live incoming counter-offers from nearby transit operators, and commit to locked negotiation limits.
- **Algorithmic Bidding Floor:** Integrated Mapbox routines compute true street-network telemetry distance. The engine calculates and enforces a strict baseline price floor (e.g., ₦150/km). The interface automatically blocks the submission of initial consumer offers falling beneath this cost boundary.
- **Live Radar Tracking:** High-speed proximity streaming with smooth marker vector interpolation, rendering continuous bike icons over map views via live WebSocket connection nodes.
- **Rider Assessment & Feedback:** Immediate visibility into bidding transit operators' verified history metrics, enforcing a mandatory 1-to-5 star rating execution post-trip completion.

3.2 Dispatch Rider Application (Flutter)

- **Automated Onboarding (KYC):** Rapid, paperless Identity verification processing Driver's License and NIN validation streams combined with biometric facial matching queries managed via dedicated third-party gateways (Smile ID / Prembly).
- **Bidding Engine Dashboard:** Asynchronous streaming consumer delivery queues, enabling operators to instantly claim base offers or submit counter-proposals to active bidding rooms.
- **Background Geolocation:** Low-power, high-durability native background GPS tracking services ensuring persistent data streaming across active delivery tasks.

3.3 Web Dashboard (Next.js)

- **Super Admin Portal:** Absolute ecosystem oversight, dispatch performance telemetry analytics, dispute settlement panels, global bidding floor adjustments, and commission ledger monitors.
- **Business Vendor Portal:** B2B multi-tenant architecture enabling brick-and-mortar entities (e.g., local eateries) to manage catalog menu listings, track real-time delivery orders, and view split revenue visualizations.

3.4 Commission & Escrow Engine

- **Automated Platform Fee Deductions:** The core server architecture maintains transactional hooks to automatically capture a set platform commission (e.g., 10-15%) upon successful bid closure.
- **Escrow Settlement Ledger:** Digital transaction sums (processed via Paystack/Monnify) are held inside an automated sandbox escrow state. Funds are programmatically unlocked and routed to the rider's platform payout wallet only upon unambiguous digital delivery confirmation by the end customer.

4. Technical Architecture & DevOps Tracks

To support absolute operational modularity, core platform software is decoupled from the underlying environment infrastructure layers. Core functional binaries remain fully identical across both paths, allowing the core code to remain environment-agnostic.

- **Frontend/Mobile Ecosystem:** Cross-platform Flutter (Dart) codebase for dual iOS and Android mobile binaries. Responsive Next.js (React/TypeScript) frameworks for web portals.
- **Backend Microservices Engine:** Asynchronous, event-driven runtime layers (Python FastAPI or Node.js) built specifically to maintain high-throughput WebSocket handshakes and stateful session routing.
- **Data Strategy & Geo-Layers:** Relational PostgreSQL clusters paired with PostGIS geospatial engines to execute coordinate distance tracking. Fast in-memory Redis clusters act as the Pub/Sub messaging pipeline for the WebSocket bidding instances.
- **Ecosystem Integrations:** Mapbox Core SDK for tile and path vector rendering, Paystack/Monnify for merchant banking, Termii for SMS OTP distributions, and Firebase Cloud Messaging (FCM) for push alerting channels.
- **Cloud Deployment Infrastructure Alignment:** Final deployment tracking maps to two selectable paths. Code is built inside standardized software containers configured to deploy either over a *Custom Self-Hosted K3s Orchestration Track* (Option 1 - featuring an upfront custom GitHub automation runner pipeline cluster) or over a *Managed Public Cloud Track* (Option 2 - mapping infrastructure builds to AWS/GCP managed database backends and recurring pipeline utility billing).

5. Operational Guardrails & Unhappy Paths

5.1 Urban Transit Anomaly Handling

- **Rider Absconds / Network Dropout:** If an active task tracking stream goes quiet with no GPS updates for a continuous 10-minute window, the engine flags the transaction, triggers an alert flag on the Super Admin panel, and routes a notification choice back to the customer to request order replacement.
- **Post-Acceptance Cancellations:** If an operator drops an order after an explicit bid lock, the customer is immediately reassigned back to the active Bidding Room. Recurring cancellations degrade the rider's visibility prioritization metrics inside upcoming bidding queues.
- **Customer No-Show:** Upon arrival within the geofenced drop-off radius, an automated 10-minute countdown timer fires. If the customer is unreachable throughout this block, the rider can log the trip as "Customer Unavailable," freeing the order lock without penalty.

5.2 Cellular Dead-Zone Recovery (Network Resilience)

- **Local In-Memory Queueing:** To resolve local network connectivity failures along Port Harcourt municipal grid paths, the Flutter mobile apps deploy Hive/Isar local data storage layers.
- **Telemetry Batch Syncing:** When an active mobile connection drops, raw geospatial coordinate logs are cached locally on the device hardware. Once a stable cellular handshake is recovered, the client application executes a bulk, ordered payload upload to update the centralized state machine cleanly.

6. Security & Fraud Mitigation

- **Collusion Tracking Arrays:** The platform continuously logs cancellation profiles matching paired riders and consumers. If a specific pairs exhibits recurring bid acceptance followed by immediate cancellation (indicating fee evasion to bypass platform splits), both accounts are marked for an automated background shadowban pending manual audit.
- **Geospatial Anti-Spoof Array:** The client applications run strict jailbreak, rooting, and Android Mock Location API validation checks. The server logic applies impossible coordinate check limits, dropping incoming packet requests showing speed metrics that violate realistic velocity bounds.

7. Legal & NDPR Compliance

- **Data Minimization Frameworks:** Biometric and official state identity validations occur directly via dedicated third-party webhook integrations. Raw citizen identity media files are completely discarded post-verification and never touch permanent Elastro Cloud disk storage arrays.
- **Data Deletion Protocols:** Users retain explicit access to a native "Delete Account" button within client apps, which executes a permanent obfuscation array over personally identifiable information (PII) while preserving fully anonymized financial records for statutory tax audits.
- **Explicit Consent:** Geolocation tracing and background system alerts demand explicit user permission checks with granular consumer disclosures.

8. Key Performance Indicators (KPIs)

- **Bid Resolution Pipeline Speed:** 85% of live consumer request queues resolve into a confirmed, accepted rider bid within 60 seconds.
- **Order Fulfillment Ratios:** At least 90% of accepted negotiation bid contracts execute to completion without mid-trip termination.
- **Ecosystem Core Availability:** Target standard of 99.9% application uptime over the WebSocket channels and core bidding APIs.

9. Execution Timeline (12-Week Sprint Matrix)

Phase & Target Block	Key Architectural Deliverables
Phase 1: Architecture & Data Layer (Weeks 1-2)	Provision target environment containers (Self-hosted K3s private array or public cloud AWS/GCP targets). Deploy relational PostgreSQL databases, extend PostGIS geospatial schemas, and verify Redis Pub/Sub memory structures. Formulate schema models for core Bids, User profiles, Reviews, and Audit Ledgers.
Phase 2: Core API & Bidding Engine (Weeks 3-5)	Engineer secure REST API routes for authentication and session processing. Construct stateful asynchronous WebSocket channels linking active Redis bidding streams. Wire functional webhooks for verification KYC gateways and Paystack/Monnify escrow splits.
Phase 3: Flutter Client Applications (Weeks 5-8)	Build cross-platform customer and rider applications. Code Termii OTP verification logic, embed real-time interactive bidding feeds, and compile localized Mapbox radar mapping arrays. Inject local Hive/Isar data stores to execute background cellular dead-zone coordinate caching.
Phase 4: Next.js Dashboards (Weeks 8-10)	Scaffold Next.js TypeScript frameworks. Implement role-based route restriction rules. Build corporate tracking dashboards for Super Admin fare floors, split commission tracking engines, and multi-tenant catalog tools for partner commercial vendors.
Phase 5: Field Verification & Launch (Weeks 11-12)	Execute high-concurrency stress testing over the WebSocket engines to simulate peak loads. Conduct multi-network physical route testing across Port Harcourt sectors to confirm cell drop recovery logic. Submit cryptographically signed release binaries to Apple App Store and Google Play Store systems.